HDOC Day-46

C Programming Assignment

Problem Statement

Write a C program that reads the size and elements of a variable-length integer array nums[]. The array may contain duplicates. Pass the array by address to Bubble_sort() to sort it in ascending order. Pass the sorted array by pointer to occurrence(), which reads a target integer, finds the first and last occurrences, and displays their values and zero-based indices. Print -1, -1 if absent.

Algorithm

1. Read array size n and n integer elements.
2. Pass nums and n to Bubble_sort() using the array address.
3. Compare adjacent elements and swap when out of ascending order.
4. Pass the sorted array to occurrence() using a pointer.
5. Read target; initialize first and last indices to -1.
6. Scan the array; record the first match and update the last match.
7. Print the first and last occurrence or -1, -1 if no match.

C Program

```
#include <stdio.h>

void Bubble_sort(int *arr, int n)
{
    int i, j, temp;
    for(i = 0; i < n - 1; i++)
    {
        for(j = 0; j < n - i - 1; j++)
        {
            if(arr[j] > arr[j + 1])
            {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

void occurrence(int *arr, int n)
{
    int target, first = -1, last = -1, i;
    printf("Enter target: ");
    scanf("%d", &target);
    for(i = 0; i < n; i++)
    {
        if(arr[i] == target)
        {
            if(first == -1)
                first = i;
            last = i;
```

```
        }
    }
    if(first == -1)
        printf("-1, -1\n");
    else
    {
        printf("First occurrence: %d at index %d\n", arr[first], first);
        printf("Last occurrence: %d at index %d\n", arr[last], last);
    }
}

int main()
{
    int n, i;
    printf("Enter size of array: ");
    scanf("%d", &n);
    if(n <= 0)
    {
        printf("Invalid size\n");
        return 0;
    }
    int nums[n];
    printf("Enter array elements: ");
    for(i = 0; i < n; i++)
        scanf("%d", &nums[i]);
    Bubble_sort(nums, n);
    printf("Sorted array: ");
    for(i = 0; i < n; i++)
        printf("%d ", nums[i]);
    printf("\n");
    occurrence(nums, n);
    return 0;
}
```

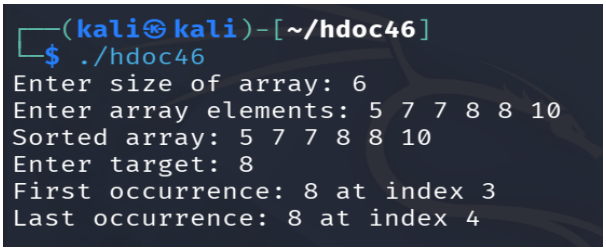
Test Cases

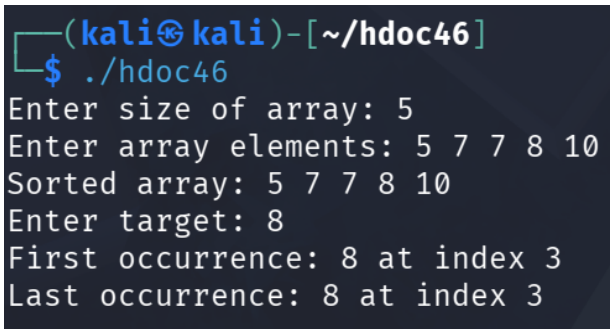
Array	Target	Expected result
5 7 7 8 8 10	8	First: 8 at index 3; Last: 8 at index 4
5 7 7 8 10	8	First: 8 at index 3; Last: 8 at index 3
5 7 7 8 8 10	6	-1, -1

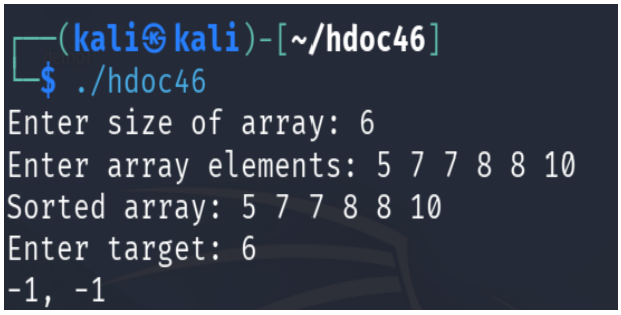
Compilation



Execution







Result

The program sorts the array using Bubble Sort and correctly identifies the first and last indices of the target, including duplicate values and the case where the target is absent.